Companion to the UI/UX Design Guide | Course AI203

NimbusHost

UI / UX in Code

How the design becomes a working React interface: design tokens, role-aware navigation, reusable components, the launch wizard, data fetching against the Express API, error mapping and accessibility.

React + Vite | CSS variables | React Router | TanStack Query

src/ui/StatusBadge.jsx

```
const TONE = {
  running: 'ok',   pending: 'warn',  stopped: 'mute',
  paid: 'ok',      issued: 'brand',  overdue: 'danger',
  urgent: 'danger', high: 'warn', low: 'mute',
};

export function StatusBadge({ value }) {
  return (
    <span className={`badge badge--${TONE[value]}`}>
      <i className="dot" aria-hidden />{value}
    </span>
  );
}

/* One component, one colour map: every status
   in the app looks and reads the same. */
```

Starter code written from the blueprint and design guide. Illustrative and not yet run against the real API.

OVERVIEW

From design principle to code mechanism

Each of the six design principles has one place in the codebase that enforces it, so the behaviour stays consistent as screens are added.

Design principle	How code enforces it	Where
Clarity over density	DataTable shows 5 to 7 columns; row details open in <Drawer>	`ui/Table.jsx`, `ui/Drawer.jsx`
Price before commitment	Wizard queries /pricing on every size change and renders the cost panel	`pages/LaunchWizard.jsx`
Status at a glance	One StatusBadge with a single tone map for instance, invoice, ticket	`ui/StatusBadge.jsx`
Safe by design	TerminateDialog needs the typed name; can() hides forbidden actions	`ui/ConfirmDialog.jsx`, `auth/can.js`
Role-aware surfaces	NAV map per role, RequireRole route guard, HOME redirect after login	`nav.js`, `auth/RequireRole.jsx`
Errors that teach	Backend maps database errors to {code, msg, hint}; client shows a banner	`be: middleware/errors.js`

Suggested folder structure (frontend/)

```
frontend/src/
  styles/      tokens.css  base.css  themes.css
  ui/          Button  StatusBadge  Table  Drawer  Toast  QuotaBar  ConfirmDialog  Banner  Skeleton
  auth/        AuthProvider.jsx  RequireRole.jsx  can.js
  layout/      Shell.jsx  Sidebar.jsx  Topbar.jsx  BottomTabs.jsx
  pages/       Dashboard  LaunchWizard  Instances  Volumes  Network  Invoices  Tickets
  pages/admin/ Capacity  Pricing  Audit  Consistency  PerfLab  Reports
  nav.js       role -> sidebar items
  api.js       fetch wrapper: JWT, retry on deadlock, friendly errors
  App.jsx      routes + providers
```

Stack from the blueprint

- **React (Vite)** — UI and fast dev server
- **React Router** — role-guarded routes
- **Recharts** — cost and utilization charts
- **TanStack Query** — fetching, polling, caching (added)
- **Lucide icons** — 1.5 px icon set (added)
- **Express + mysql2** — JSON API over stored procedures

Libraries marked "added" are not in the blueprint. They are optional conveniences; plain fetch with useEffect also works if the team wants to stay minimal.

FOUNDATION

Design tokens as CSS variables

Every colour, radius and space value lives in one file. Components never hard-code a hex value, which is what makes dark mode and re-theming a one-file change.

src/styles/tokens.css

```
:root {
  /* brand and status */
  --brand: #2F6BFF;  --accent: #7C5CFF;  --sky: #5AC8FA;
  --ok: #19C37D;  --warn: #F5A524;  --danger: #F04461;  --mute: #94A3B8;
  /* surfaces */
  --bg: #F5F7FB;  --card: #FFFFFF;  --ink: #1B2437;  --muted: #6B7690;  --line: #E2E8F2;  --nav: #0B1B3A;
  /* shape and 4-pt spacing */
  --r-sm: 6px;  --r-md: 12px;  --r-pill: 999px;
  --s-1: 4px;  --s-2: 8px;  --s-3: 12px;  --s-4: 16px;  --s-6: 24px;  --s-8: 32px;
  --shadow-card: 0 1px 2px rgba(11,27,58,.06), 0 6px 18px rgba(11,27,58,.06);
  /* type */
  --font-ui: "Inter", system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, monospace;
}
```

src/styles/themes.css: dark mode swaps values only

```
[data-theme="dark"] {
  --bg: #0B1224;  --card: #141E3A;  --ink: #E6ECFA;  --muted: #93A1C4;  --line: #24315A;
}
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) { --bg: #0B1224;  --card: #141E3A;  --ink: #E6ECFA;  --muted: #93A1C4;  --line: #24315A; }
}
```

src/styles/base.css: type scale and card

```
body { margin: 0; background: var(--bg); color: var(--ink); font: 400 14px/22px var(--font-ui); }
h1 { font: 700 24px/32px var(--font-ui); }   h2 { font: 700 18px/26px var(--font-ui); }
.mono { font-family: var(--font-mono); font-size: 13px; }   /* IPs, instance ids, invoice numbers */
.money { text-align: right; font-variant-numeric: tabular-nums; }
.card { background: var(--card); border: 1px solid var(--line); border-radius: var(--r-md); box-shadow: var(--shadow-card);
    padding: var(--s-4); }
```

src/ui/theme.js: remember the choice

```
export function setTheme(t) {
  document.documentElement.dataset.theme = t;
  try { localStorage.setItem("theme", t); } catch { /* private mode: ignore */ }
}
export const initTheme = () => setTheme(localStorage.getItem("theme") ?? "light");
```

Design rule carried into code: currency uses the .money class (right aligned, tabular digits) and machine values use .mono, matching the money formatting rule in the design guide.

ROLE-AWARE UI

One shell, five experiences

The blueprint enforces roles at two levels. The UI adds a third layer for experience: it never shows what a role cannot use, and the API still re-checks every request.

src/nav.js: sidebar items per role

```
export const NAV = {
  customer:    [['/', 'Dashboard'], ['/instances', 'Instances'], ['/volumes', 'Volumes'], ['/network', 'Network & DNS'],
                ['/invoices', 'Invoices'], ['/usage', 'Usage'], ['/tickets', 'Tickets'], ['/team', 'Team & API keys']],
  billing:     [['/billing', 'Overview'], ['/billing/invoices', 'All invoices'], ['/billing/aging', 'Aging report']],
  support:     [['/queue', 'Ticket queue'], ['/my-tickets', 'My tickets'], ['/customers', 'Customers']],
  super_admin: [['/capacity', 'Capacity'], ['/pricing', 'Pricing'], ['/audit', 'Audit log'], ['/perf', 'Performance Lab']],
};
export const HOME = { customer: '/', billing: '/billing', support: '/queue', super_admin: '/capacity' };
```

src/auth/RequireRole.jsx: route guard (covers tests T22, T23)

```
export default function RequireRole({ allow, children }) {
  const { user, loading } = useAuth();
  if (loading) return <Skeleton />;
  if (!user) return <Navigate to="/login" replace state={{ expired: true }} />;   // 401: back to login
  if (!allow.includes(user.role)) return <Forbidden />;                          // 403 page with way home
  return children;
}
```

src/auth/can.js: hide actions, do not grey them out

```
const PERMS = {
  owner:  ['launch', 'start', 'stop', 'resize', 'terminate', 'pay', 'manageTeam'],
  member: ['launch', 'start', 'stop'],
};
export const can = (user, action) => PERMS[user.memberRole]?.includes(action) ?? false;

// in a row menu:  {can(user, "terminate") && <MenuItem onClick={askTerminate}>Terminate</MenuItem>}
```

src/styles/shell.css: sidebar becomes bottom tabs on phones

```
.shell { display: grid; grid-template-columns: 220px 1fr; min-height: 100vh; }
.sidebar { background: var(--nav); }
@media (max-width: 768px) {
  .shell { grid-template-columns: 1fr; padding-bottom: 56px; }
  .sidebar { position: fixed; bottom: 0; left: 0; right: 0; height: 56px; display: flex; justify-content: space-around; }
}
```

T26 (tenant isolation) is an API concern: GET /instances/:id must check org_id against the JWT and answer 404. The UI only needs a NotFound page to show when it does.

REUSABLE PARTS

Components that carry the design rules

Three components do most of the design work: StatusBadge for status, QuotaBar for limits and ConfirmDialog for destructive actions.

src/ui/StatusBadge.jsx + CSS: colour, dot and text together, never colour alone

```
const TONE = { running: "ok", pending: "warn", stopped: "mute", suspended: "warn", terminated: "danger",
               draft: "mute", issued: "brand", paid: "ok", overdue: "danger", void: "mute",
               low: "mute", normal: "brand", high: "warn", urgent: "danger" };
export const StatusBadge = ({ value }) => (
  <span className={`badge badge--${TONE[value]}`}><i className="dot" aria-hidden />{value}</span>
);

.badge { display: inline-flex; align-items: center; gap: 6px; padding: 2px 10px; border-radius: var(--r-pill); font: 600 12px
    var(--font-ui); }
.badge--ok { color: var(--ok); background: color-mix(in srgb, var(--ok) 16%, transparent); }  /* repeat per tone */
.badge .dot { width: 6px; height: 6px; border-radius: 50%; background: currentColor; }
```

src/ui/QuotaBar.jsx: amber at 80 percent, red at 95 percent

```
export function QuotaBar({ label, used, total }) {
  const pct = Math.min(100, (used / total) * 100);
  const tone = pct >= 95 ? "danger" : pct >= 80 ? "warn" : "brand";
  return (
    <div className="quota">
      <div className="quota__row"><span>{label}</span><b>{used} / {total}</b></div>
      <div role="progressbar" aria-label={label} aria-valuenow={used} aria-valuemax={total} className="quota__track">
        <span className={`quota__fill quota__fill--${tone}`} style={{ width: `${pct}%` }} />
      </div>
      {pct >= 80 && <small>You are close to your plan limit.</small>}
    </div>
  );
}
```

src/ui/ConfirmDialog.jsx: type the name to terminate (design guide, page 8)

```
export function TerminateDialog({ instance, onConfirm, onClose }) {
  const [typed, setTyped] = useState("");
  const ok = typed === instance.name;
  return (
    <dialog open aria-labelledby="t-title">
      <h2 id="t-title">Terminate {instance.name}?</h2>
      <p>The server stops, its IP is released and volumes are detached. This cannot be undone.</p>
      <input aria-label="Type the server name" value={typed} onChange={(e) => setTyped(e.target.value)} />
      <button className="btn btn--danger" disabled={!ok} onClick={onConfirm}>Terminate</button>
      <button className="btn btn--ghost" onClick={onClose}>Cancel</button>
    </dialog>
  );
}
```

CORE FLOW

Launch wizard implementation

The wizard keeps one state object across four steps, asks the API for live prices and capacity, and calls sp_launch_instance through POST /instances only on the final step.

src/pages/LaunchWizard.jsx: state, live price and capacity

```
const [step, setStep] = useState(0);                          // 0 region, 1 size, 2 image, 3 review
const [form, setForm] = useState({ regionId: null, typeId: null, imageId: null, name: "" });

const price = useQuery({
  queryKey: ["price", form.typeId],
  queryFn: () => api.get(`/pricing?type=${form.typeId}`),   // temporal price valid today
  enabled: !!form.typeId,
});
const capacity = useQuery({ queryKey: ["cap", form.regionId], queryFn: () =>
    api.get(`/hosts/capacity?region=${form.regionId}`), enabled: !!form.regionId });

const perHour = price.data?.price_per_hour;
const perMonth = perHour ? (perHour * 730).toFixed(2) : null;      // 730 h is the average month length
```

Capacity chip: prevent "No capacity" instead of reporting it

```
// thresholds are a design suggestion; tune them against real host data
function capacityLevel(freeVcpu, totalVcpu) {
  const free = freeVcpu / totalVcpu;
  return free > 0.30 ? "plenty" : free > 0.10 ? "limited" : "full";
}
// "full" regions are disabled in step 1 and the wizard suggests the next region with room.
```

Final step: launch, keep the form on failure

```
const qc = useQueryClient();
const launch = useMutation({
  mutationFn: () => api.post("/instances", form),                   // runs CALL sp_launch_instance(...)
  onSuccess: () => { toast.success("Server launched"); qc.invalidateQueries({ queryKey: ["instances"] });
      navigate("/instances"); },
  onError: (e) => setBanner({ msg: e.msg, hint: e.hint }),             // form state is untouched, user fixes and retries
});

<button className="btn btn--success" disabled={launch.isPending || !form.name} onClick={() => launch.mutate()}>
  {launch.isPending ? <Spinner /> : "Launch server"}
</button>
```

> Name is validated on blur against UNIQUE(org_id, name) by calling GET /instances?name=. If the check endpoint is skipped, the 409 from the database is shown inline under the field instead.

DATA LAYER

Which screen reads which database object

Screens never talk to tables directly. Each one calls a blueprint endpoint, which reads a view or calls a stored procedure. Admin endpoints marked (suggested) are not in the blueprint list.

Dashboard KPIs	`GET /instances, GET /usage`	`v_running_instances, v_org_monthly_cost`
Cost chart	`GET /usage`	`daily_usage_summary`
Quota ring	`GET /orgs (suggested: quota fields)`	`fn_org_quota_left, account_tiers`
Launch wizard	`GET /instance-types /images /pricing /hosts/capacity; POST /instances`	`fn_hourly_price, v_host_capacity, sp_launch_instance`
Instance actions	`POST /instances/:id/{start,stop,resize,terminate}`	`sp_resize_instance, sp_terminate_instance`
Invoices and pay	`GET /invoices, /invoices/:id; POST /invoices/:id/pay`	`v_overdue_invoices, sp_apply_payment`
Ticket queue and search	`GET /tickets, /tickets/search?q=`	`v_ticket_sla, FULLTEXT on ticket_messages`
Capacity heat map	`GET /hosts/capacity`	`v_host_capacity, fn_host_utilization`
Audit viewer	`GET /audit`	`audit_log (read only)`
Consistency checker	`POST /admin/consistency-check`	`sp_check_consistency`
Performance Lab	`POST /admin/perf/run (suggested)`	`benchmark queries, invisible index toggle`
Reports R1 to R20	`GET /reports/R1 ... R20`	`views and report queries from section 10`

Live dashboards: polling every 10 seconds, no flicker

```
const capacity = useQuery({
  queryKey: ["capacity"],
  queryFn: () => api.get("/hosts/capacity"),
  refetchInterval: 10_000,          // blueprint: polling every few seconds
  placeholderData: keepPreviousData, // old numbers stay visible while refreshing
});
if (capacity.isPending) return <Skeleton rows={6} />;      // first load only
```

Avoid N+1 requests: the instances page should call one endpoint that already joins organization, type, host and IP (v_running_instances), not one call per row.

FEEDBACK

Turning database errors into helpful messages

Raw SQL text never reaches the user. The backend maps each error to a stable code, a message and a hint; the client decides whether to show a banner, a toast or an inline field error.

backend/src/middleware/errors.js

```
const FRIENDLY = [
  [/No capacity/i,       { code: "NO_CAPACITY", msg: "That region is full right now.", hint: "Try another region or a smaller
      size." }],
  [/tier limit|quota/i,  { code: "QUOTA", msg: "You reached your plan limit.", hint: "Stop a server or upgrade your plan."
      }],
  [/suspended/i,         { code: "SUSPENDED", msg: "Your account is paused.", hint: "Pay the overdue invoice to reactivate."
      }],
];

export function dbErrorHandler(err, req, res, next) {
  if (err.errno === 1213) return res.status(503).json({ code: "RETRY", msg: "Something clashed. Trying again..." });
      // deadlock
  if (err.errno === 1062) return res.status(409).json({ code: "DUPLICATE", msg: "That name or value is already used." });
      // UNIQUE
  if (err.errno === 1451) return res.status(409).json({ code: "IN_USE", msg: "This still has items attached.", hint: "Remove
      them first." }); // FK RESTRICT
  if (err.errno === 3819) return res.status(400).json({ code: "INVALID", msg: "One of the values is not allowed." });
      // CHECK
  if (err.sqlState === "45000") {                                          // SIGNAL from procedures and triggers
    const hit = FRIENDLY.find(([re]) => re.test(err.sqlMessage));
    if (hit) return res.status(422).json(hit[1]);
  }
  res.status(500).json({ code: "UNKNOWN", msg: "Something went wrong. Please try again." });   // never send err.sqlMessage
}
```

frontend/src/api.js: token, session expiry and automatic retry

```
export async function request(path, opts = {}, attempt = 1) {
  const res = await fetch(`${API}${path}`, { ...opts, headers: { "Content-Type": "application/json", Authorization: `Bearer
      ${getToken()}`, ...opts.headers } });
  if (res.status === 401) { clearToken(); window.location.assign("/login?expired=1"); return; }             // T23
  const body = await res.json().catch(() => ({}));
  if (body.code === "RETRY" && attempt < 3) return request(path, opts, attempt + 1);                         // T20 retry on
      1213
  if (!res.ok) throw Object.assign(new Error(body.msg), body);   // e.msg and e.hint feed <Banner>
  return body;
}
```

Where each message appears

```
// inline  -> field errors (DUPLICATE name, INVALID)      <FieldError />
// toast   -> success and soft warnings, 5 s                 toast.success / toast.warn
// banner  -> page-level problems that need an action      <Banner tone="danger" hint={e.hint} />
// dialog  -> irreversible actions only                      <TerminateDialog />
```

QUALITY

Accessibility, charts and the Performance Lab

The details that make the interface usable for everyone and the demo convincing to examiners.

src/styles/a11y.css

```
:focus-visible { outline: 2px solid var(--brand); outline-offset: 2px; }             /* visible focus ring */
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }
@media (pointer: coarse) { .btn, .icon-btn, .row-action { min-height: 44px; min-width: 44px; } }   /* touch targets */
@media (max-width: 768px) {
  table, thead, tbody, tr, td { display: block; }                /* table becomes stacked cards */
  td::before { content: attr(data-label); font-weight: 600; color: var(--muted); }
}
/* toast container:  <div role="status" aria-live="polite"> ...   modals: focus trap, Esc closes */
```

Cost chart with Recharts, coloured from tokens so dark mode works

```
<ResponsiveContainer width="100%" height={220}>
  <AreaChart data={daily}>
    <XAxis dataKey="date" tick={{ fill: "var(--muted)", fontSize: 11 }} />
    <YAxis tick={{ fill: "var(--muted)", fontSize: 11 }} />
    <Tooltip />
    <Area dataKey="cost" stroke="var(--brand)" fill="var(--brand)" fillOpacity={0.12} strokeWidth={2} />
  </AreaChart>
</ResponsiveContainer>
```

backend: Performance Lab endpoint (admin only, whitelisted queries)

```
const BENCH = { Q3: { sql: "SELECT invoice_id, org_id, due_date FROM invoices WHERE status = ? AND due_date < ?", params:
    ["issued", "2026-10-01"], index: "idx_status_due", table: "invoices" } };

router.post("/admin/perf/run", requireRole("super_admin"), async (req, res) => {
  const q = BENCH[req.body.queryId];                       // only known ids: never run SQL typed by the user  (T24)
  if (!q) return res.status(400).json({ code: "INVALID", msg: "Unknown benchmark." });
  const state = req.body.indexVisible ? "VISIBLE" : "INVISIBLE";
  await pool.query(`ALTER TABLE ${q.table} ALTER INDEX ${q.index} ${state}`);          // names come from BENCH, not from the
      request
  const [rows] = await pool.query("EXPLAIN ANALYZE " + q.sql, q.params);
  res.json({ plan: Object.values(rows[0])[0] });
});
```

> The side-by-side cards in the design guide call this endpoint twice (index invisible, then visible) and render the returned plan text in the dark code panel. Show real numbers only.

WRAP UP

Build order and verification

A suggested order that gets a demo-able interface early, then adds the admin features.

1. **Tokens + shell**
   tokens.css, themes.css, Shell, Sidebar, Topbar
2. **Auth + roles**
   AuthProvider, RequireRole, NAV, login, 401 and 403 pages
3. **UI kit**
   Button, StatusBadge, Table, Drawer, Toast, QuotaBar, ConfirmDialog
4. **Dashboard + instances**
   KPIs, cost chart, instance table, row menu
5. **Launch wizard**
   live price, capacity chips, error banner
6. **Billing + tickets**
   invoice detail and pay, aging chart, queue with SLA colours
7. **Admin + Perf Lab**
   heat map, audit diff, consistency, performance lab

UI checks tied to the blueprint test table

T22	Customer opening an admin URL sees the 403 page	RequireRole
T23	Expired token returns to login	api.js 401 branch
T24	SQL in the ticket search box is plain text	parameterised API, no raw SQL
T25	Wizard happy path and readable error path	LaunchWizard + Banner
T26	Other tenant instance URL shows Not found	API org check + NotFound page
T20	Deadlock error retried automatically	api.js retry

Status of this guide: the code shown is a starting point derived from the blueprint and the design guide. Run it against the real API, adjust names to your final schema, and capture screenshots for the report.